Functions: Deep Dive

Why the implementation looks like this

Gregory S. DeLozier, PhD

Named Form Is Pure Sugar

```
def parse_function_tail(tokens):
    # "(" [ identifier { "," identifier } ] ")" block
    ...
    node = {"tag": "function", "parameters": parameters, "body": body}
    return node, tokens
```

`function_expression` and `function_definition` both call this one helper right after consuming `"function"`. The only difference is whether an identifier came first.

Disambiguating `function`

```
if tokens[0]["tag"] == "function" and tokens[1]["tag"] == "identifier":
    return parse_function_definition(tokens)
...
return parse_expression(tokens)
```

One token of lookahead decides it: an identifier after `function` means the named sugar; anything else falls through to the ordinary expression path.

A Call Shares the Suffix Loop, Not the `complex` Node

```
while tokens[0]["tag"] in ("[", ".", "("):
    if tokens[0]["tag"] == "[":
        ...
        node = {"tag": "complex", "base": node, "index": index}
    elif tokens[0]["tag"] == ".":
        ...
        node = {"tag": "complex", "base": node, "index": index}
    else:
        arguments, tokens = parse_call_arguments(tokens)
        node = {"tag": "call", "callee": node, "arguments": arguments}
```

`(args)` is the third suffix kind in the same loop `[` and `.` already use, but it builds its own `"call"` node, not another `"complex"` one.

Dynamic Scoping: Retain Nothing But Parameters and Body

```
if ast["tag"] == "function":
    global function_count
    function_count += 1
    return {FUNCTION_KEY: function_count,
            "parameters": ast["parameters"],
            "body": ast["body"]}, None
```

That is the whole representation. No reference to where it was defined.

A Non-String Key Makes a Function Value Unambiguous

```
FUNCTION_KEY = None  # not a string -- no object literal can ever write it


def is_function(value):
    return type(value) is dict and FUNCTION_KEY in value
```

The evaluator already requires every object key to be a string; a key that is not one is reserved for free.

Where Leaning on Python's Types Falls Short

```
# every earlier chapter: ask Python what kind of value this is
type(value) is dict    # a Vertex object?
type(value) is list    # a Vertex list?

# this chapter: ask a reserved key instead
is_function(value)     # a Vertex function?
```

Python's types answer "what kind of Python object is this," not "what kind of Vertex value is this" – this chapter is the first time that gap mattered.

Arity Checked After Arguments, Not Before

```
arguments = []
for argument_ast in ast["arguments"]:
    value, status = evaluate(argument_ast, environment)
    if status is not None:
        return value, status
    arguments.append(value)
if len(arguments) != len(callee["parameters"]):
    raise TypeError(...)
```

Every argument is evaluated, and checked, before arity is even looked at.

A Call Is a New Status Boundary

Body's status	Call reports
`"return"`	The value, with no status
`None` (fell off the end)	`null`, with no status
`"exit"`	Passed outward unchanged
`"break"` or `"continue"`	A specific error

```
if status in ("break", "continue"):
    raise ValueError(f"'{status}' used outside a loop")
```

Inlined at both boundaries, not shared – four lines twice is not worth a helper.

Assignment Moves Into the Expression Grammar

```
def parse_expression(tokens):
    left, tokens = parse_logic_or(tokens)
    if tokens[0]["tag"] == "=":
        right, tokens = parse_expression(tokens[1:])
        return {"tag": "assign", "target": left, "expression": right}, tokens
    return left, tokens
```

Lowest precedence, right-associative: `x = y = 1` parses as `x = (y = 1)`.

Assignability Is an Evaluator Question Now

```
if target["tag"] == "identifier":
    ...
elif target["tag"] == "complex":
    ...
else:
    raise ValueError(f"Cannot assign to a {target['tag']} expression")
```

`1 = 2`, `f() = 5`, and `(x + 1) = 5` all parse. None of them evaluate.

Any Expression May Be a Statement

```
if tokens[0]["tag"] == "return":
    return parse_return_statement(tokens)
return parse_expression(tokens)
```

That is the entire fallback. No more `assignment_statement`, no more bare-call special case.

`exit` Cannot Nest, But a Call Can

```
numbers = [10, 20, 30];
function stopAt(code) { exit(code); };
print(numbers[stopAt(5)]);
```

Exits with status 5. The index is never used; the print never runs.

What Carries Forward

The suffix loop absorbed a third kind the same way it absorbed its second: widen the loop, add a node, not a parallel design.

A function value retaining only parameters and body is this chapter's one real limitation – the next chapter's closures retain more.

Statements admitting any expression exist so a bare call can be a statement; assignment becoming an expression is what that unification then required.